CEBU INSTITUTE OF TECHNOLOGY UNIVERSITY

COLLEGE OF COMPUTER STUDIES

Software Requirements Specifications

for

CapVault: A Capstone Project Tracking and Archival Management System

Change History

Version	Date	Author/s	Description
1.0	5/22/26	Sia, Fernandez, Tabanas, Taghoy	Initial SRS draft for CapVault

Table of Contents

1. Introduction

1.1. Purpose

The purpose of this document is to provide a detailed description of CapVault: A Capstone Project Tracking and Archival Management System. This Software Requirements Specification serves as a reference for the project team, adviser, developers, and stakeholders by defining the system's purpose, scope, user roles, functional requirements, external interfaces, and non-functional requirements. This document will guide the development of the system and help ensure that the proposed software aligns with the capstone project objectives, expected system workflow, and required deliverables.

1.2. Scope

CapVault is a web-based system designed to help students, advisers, and administrators manage capstone project tracking and document archiving in one application. The system allows users to log in using Google account validation, import class records from Google Sheets, manage capstone groups and deliverables, submit capstone documents through file upload or Google Drive links, preserve document versions, archive submitted files, generate SHA-256 hashes for file integrity, and retrieve records through search and dashboard reports.The system includes the following core functionalities:

- Google account validation and role-based access
- Google Sheets class record import and column mapping
- Capstone group and deliverable tracking
- Student document submission through upload or Google Drive link
- Submission status tracking and adviser remarks
- Version control for updated submissions
- PDF conversion for supported document formats
- Archival vault with metadata
- SHA-256 file integrity hashing
- Search, retrieval, dashboard summaries, and reports
- Activity logs for important user and archive actions

1.3. Definitions, Acronyms and Abbreviations

Term	Definition
Admin	The user role responsible for managing users, class records, system settings, and archive records.
Adviser	The user role responsible for monitoring assigned capstone groups, reviewing submissions, adding remarks, and updating submission status.
Student	The user role responsible for submitting capstone deliverables and viewing their group's submission history.
Capstone Project	A major academic project completed by students as part of their course requirement.

Deliverable	A required capstone output such as proposal, SRS, SDD, SPMP, final manuscript, presentation file, or source code record.
Archive	A controlled storage area where submitted capstone documents are preserved.
Metadata	Descriptive information attached to an archived document, such as project title, group code, adviser, deliverable type, version number, date submitted, archive date, and status.
Version Control	The process of saving updated submissions as separate versions instead of overwriting previous files.
RBAC	Role-Based Access Control.
OAuth2	Open Authorization 2.0, used for secure Google account login.
JWT	JSON Web Token, used for authenticated user sessions.
API	Application Programming Interface.
UI	User Interface.
DTO	Data Transfer Object.
CRUD	Create, Read, Update, Delete.
ERD	Entity Relationship Diagram.
SHA-256	Secure Hash Algorithm 256-bit, used for file integrity verification.
PDF	Portable Document Format.
SRS	Software Requirements Specification.
SDD	Software Design Description.

1.4. References

- CapVault Project Proposal, Weeks 7–8, Team 2526-sem2-it332-41.
- CapVault Review of Related Literature, Existing Solutions Analysis, Gap Identification, and Traceability Matrix, Weeks 5–6, Team 2526-sem2-it332-41.
- Google Developers. (n.d.). Using OAuth 2.0 to access Google APIs. https://developers.google.com/identity/protocols/oauth2
- Google Developers. (n.d.). Google Sheets API overview. https://developers.google.com/workspace/sheets/api/guides/concepts
- Google Developers. (n.d.). Google Drive API overview. https://developers.google.com/workspace/drive/api/guides/about-sdk
- React. (n.d.). React documentation. https://react.dev/
- Spring. (n.d.). Spring Boot documentation. https://docs.spring.io/spring-boot/index.html
- PostgreSQL Global Development Group. (n.d.). PostgreSQL documentation. https://www.postgresql.org/docs/

2. Overall Description

2.1. Product perspective

CapVault is a web-based client-server application that combines capstone project tracking and document archiving. The system will use React for the frontend, Spring Boot for the backend, and PostgreSQL for the database. It will also integrate Google APIs for account validation, Google Sheets class record import, and Google Drive document retrieval.

The system is intended to replace scattered manual workflows where capstone records are handled through separate Google Sheets, Google Drive folders, shared links, chat messages, and manual adviser checking. CapVault centralizes these activities by allowing class records, group details, deliverables, submissions, versions, archive records, and reports to be managed in one platform.

2.2. User characteristics

- Admin: The Admin manages user accounts, imports or configures class records, manages system settings, oversees archive records, and accesses overall reports. The Admin is expected to have basic knowledge of class records, Google Sheets, and system management.
- Adviser: The Adviser monitors assigned capstone groups, checks submission status, reviews submitted documents, adds remarks, and marks deliverables as under review, needs revision, approved, rejected, or final. The Adviser is expected to have basic computer literacy and knowledge of capstone requirements.
- Student: The Student submits capstone deliverables, uploads files or Drive links, views submission status, reads adviser remarks, and checks previous submission versions. The Student is expected to have basic knowledge of Google accounts and online document submission.

2.4. Constraints

- The system requires internet access for Google login, Google Sheets integration, and Google Drive retrieval.
- The system depends on valid Google API permissions and properly configured Google API credentials.
- The system will support only Admin, Adviser, and Student roles.
- The system will be limited to capstone tracking and archiving features.
- The system will not compute grades or evaluate document quality automatically.
- File upload limits may depend on server storage capacity and deployment configuration.
- PDF conversion may depend on supported file formats and document access permissions.
- Google Drive link retrieval may fail if the submitted file has restricted permissions, is deleted, or is inaccessible.
- The prototype will be tested using a selected capstone class or controlled sample dataset.
- The system must protect academic records through role-based access and secure authentication.

2.5. Assumptions and dependencies

- Users will have valid Google accounts.
- The instructor or Admin will provide a Google Sheets class record that can be imported or mapped.
- Submitted Google Drive links will be accessible to the system when retrieval is needed.
- The system will have access to Google OAuth, Google Sheets API, and Google Drive API.
- The backend server, database, and file storage will be available during system operation.
- The system will be deployed in an environment that supports React, Spring Boot, PostgreSQL, and secure file handling.
- Students and advisers will follow the required submission and review process.
- The system requirements may change if the capstone class record format, Google API access, or institutional workflow changes.

3. Specific Requirements

3.1. External interface requirements

3.1.1. Hardware interfaces

CapVault is a web-based system and does not require direct interaction with specialized hardware. Users will access the system using desktop computers, laptops, tablets, or mobile devices with a modern web browser. The server must have enough storage capacity to handle uploaded documents and archived files.

Minimum supported user device requirements:

- Internet-capable desktop, laptop, tablet, or smartphone
- Modern web browser such as Google Chrome, Microsoft Edge, Firefox, or Safari
- Stable internet connection for login, upload, retrieval, and Google API features

Server-side requirements:

- Server or hosting environment capable of running Spring Boot
- PostgreSQL database server
- File storage for uploaded and archived documents
- Secure network connection for Google API communication

3.1.2. Software interfaces

CapVault will interface with the following software and services:

- **React** will be used for the frontend user interface, including login pages, dashboards, submission forms, archive views, and report pages.
- **React Router** will be used for frontend navigation between role-based pages such as Admin dashboard, Adviser dashboard, Student submission page, archive search, and reports.
- **Spring Boot with Java 21** will be used for backend services, business logic, REST API endpoints, authentication handling, file processing, and communication with the database and Google APIs.
- **Spring Security** will be used to manage authentication, authorization, and role-based access control for Admin, Adviser, and Student users.
- **Google OAuth2** will be used for Google account validation during login.
- **JWT** may be used to manage authenticated user sessions between the frontend and backend after successful login.
- **PostgreSQL** will be used as the main database for storing user records, class records, capstone groups, deliverables, submission records, document metadata, version history, archive records, and activity logs.
- **Google Sheets API** will be used to import and synchronize class records from Google Sheets, including student names, emails, sections, group codes, adviser assignments, project titles, and related class information.
- **Google Drive API** will be used to retrieve submitted Google Drive files and export supported Google Docs formats for archival processing.

- **Java-based file handling services** will be used for upload processing, archived file storage, file naming, document retrieval, and file download handling.
- **PDF conversion library or service** will be used to convert supported document formats, such as DOCX or Google Docs, into PDF format when applicable.
- **SHA-256 hashing utility** will be used to generate and compare file hashes for archived document integrity verification.
- **Recharts or a similar charting library** may be used to display dashboard summaries such as submitted, missing, late, approved, final, and archived document counts.
- **Git and GitHub** will be used for source code version control and team collaboration.
- **Postman or a similar API testing tool** may be used to test backend endpoints during development.

3.1.3. Communications interfaces

CapVault will communicate through standard web protocols. The frontend will send requests to the backend through HTTP or HTTPS API calls. The backend will communicate with Google services through secure API requests. The database connection will be handled through backend database drivers and should not be directly exposed to users.

Communication requirements:

- HTTPS must be used during deployment to protect login and document-related data.
- API requests between the frontend and backend must use structured request and response formats such as JSON.
- Google API communication must follow Google authentication and authorization requirements.
- File upload and download requests must be handled securely.
- User sessions must expire after a defined period of inactivity.

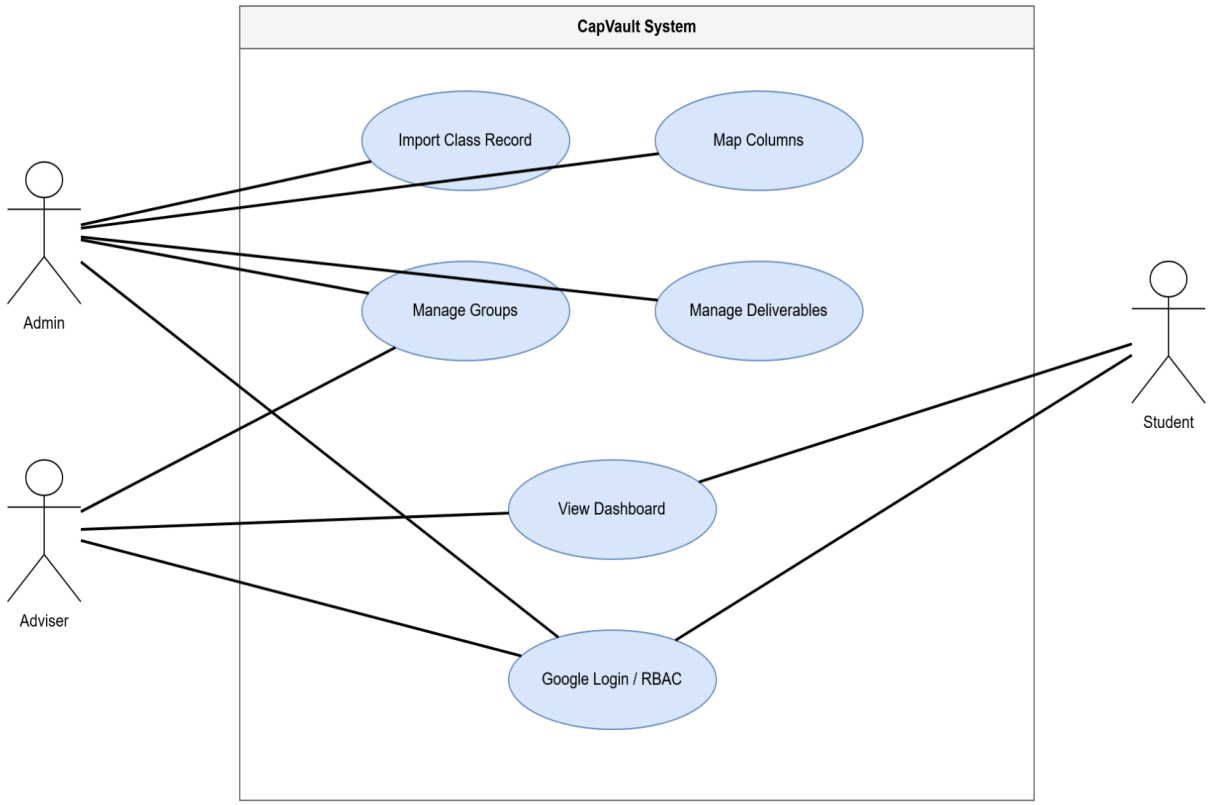
3.2. Functional requirements

Module 1: Capstone Tracking and Class Record Management

1.1 Import Class Record

Use Case Diagram:

UC - Module 1 Capstone Tracking



Use Case Description:

Use Case Name: *Import Class Record*
Primary Actor: *Admin*
Supporting System: *Google Sheets API*

Goal:
To import student, group, adviser, section, project, and capstone record data from a Google Sheets class record into CapVault.

Preconditions:
The Admin is logged in. Google API access is configured. The Google Sheets class record is accessible to the

system.

Main Flow:

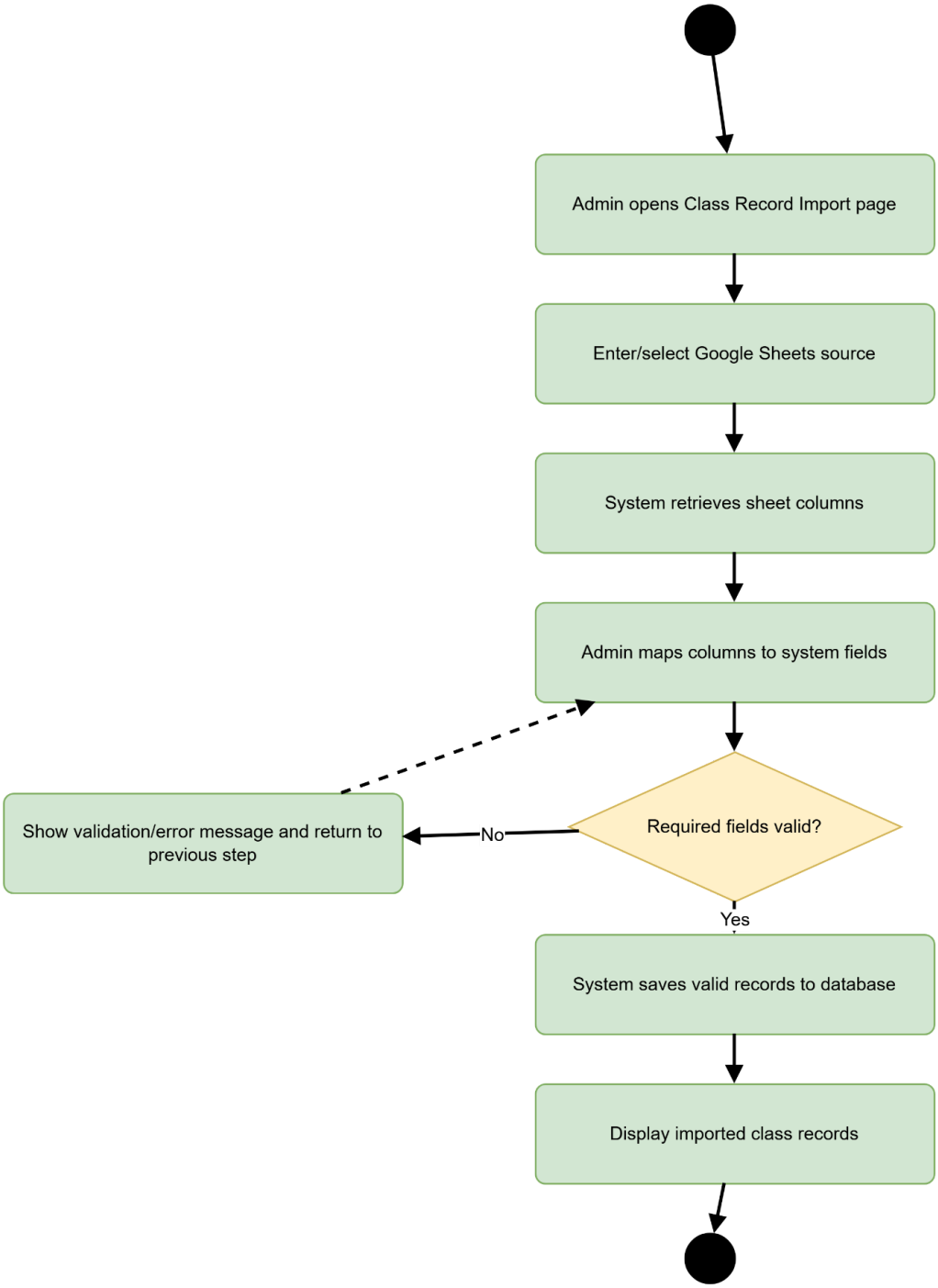
1. *The Admin opens the Class Record Import page.*
2. *The Admin enters or selects the Google Sheets source.*
3. *The system retrieves the available sheet columns.*
4. *The Admin maps spreadsheet columns to system fields such as group code, project title, software name, description, remarks, adviser/status, category, document links, student name, email, and section.*
5. *The system validates the mapped records.*
6. *The Admin confirms the import.*
7. *The system stores valid records in the database.*
8. *The system displays the imported records in the class record dashboard.*

Alternative Flow:
If required fields are missing or invalid, the system displays validation errors and asks the Admin to correct the column mapping.

Postconditions:
Valid class records are imported and become available for group tracking, submission management, and archive organization.

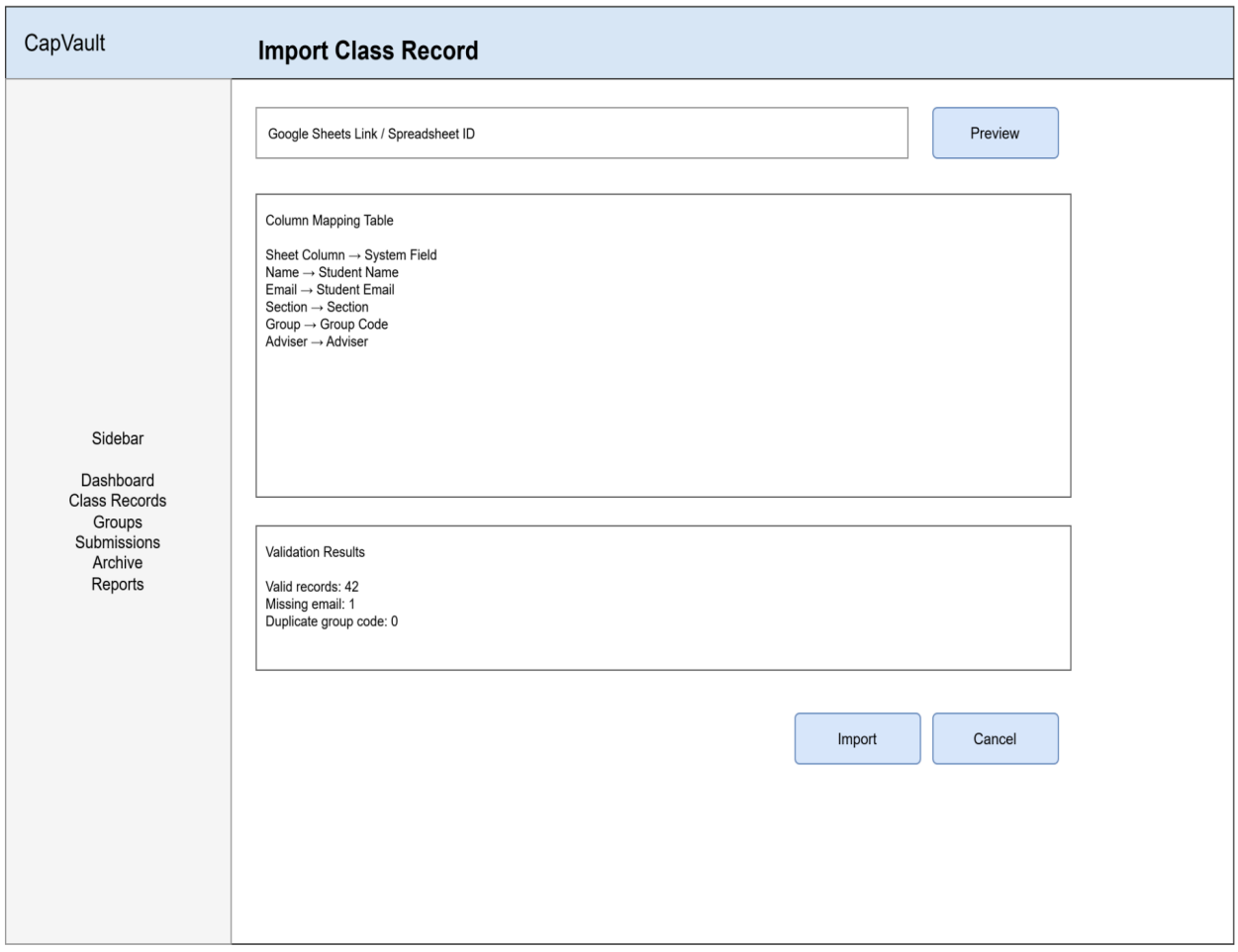
Activity Diagram:

Activity - 1.1 Import Class Record



Wireframe:

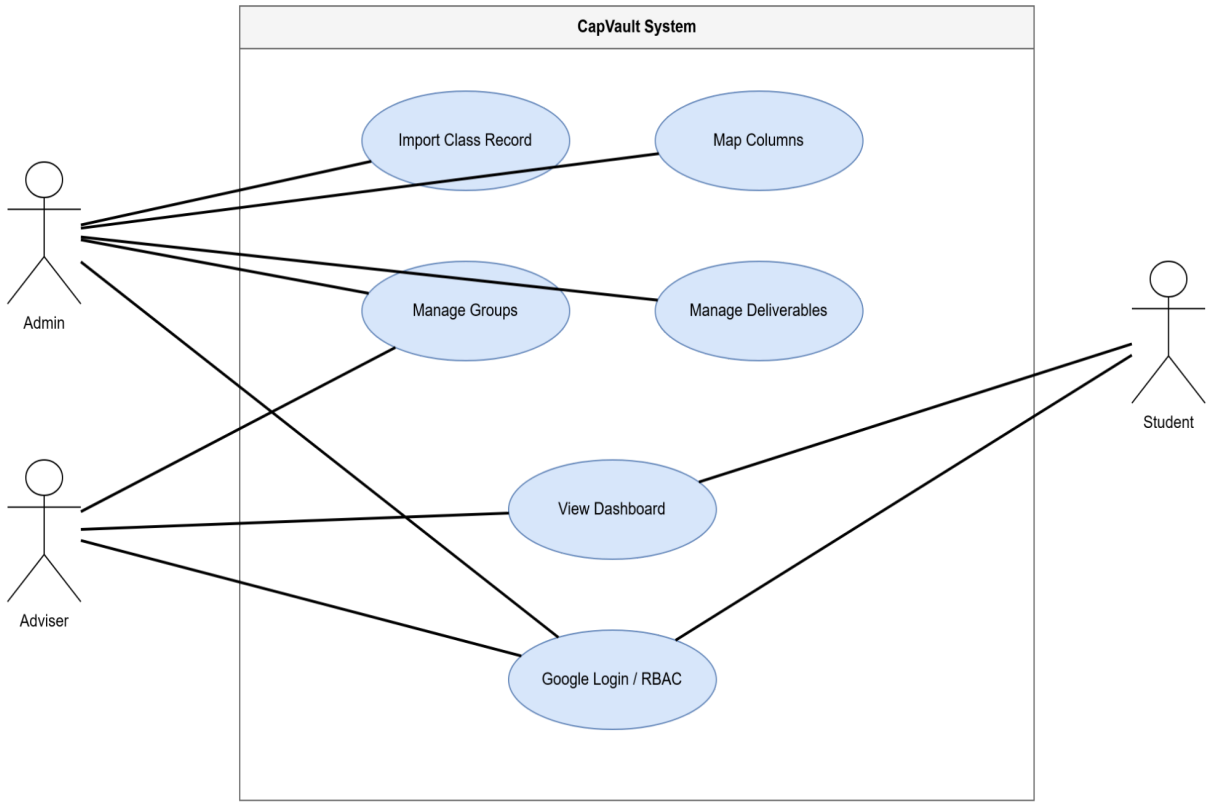
Wireframe - 1.1 Import Class Record



1.2 Manage Capstone Groups and Deliverables

Use Case Diagram:

UC - Module 1 Capstone Tracking



Use Case Description:

Use Case Name: *Manage Capstone Groups and Deliverables*
Primary Actor: *Admin*
Secondary Actor: *Adviser*

Goal:
To organize capstone groups, project titles, advisers, deliverables, deadlines, remarks, and submission statuses in one system.

Preconditions:
The Admin or Adviser is logged in. Class records have already been imported or manually created.

Main Flow:

1. *The user opens the Group Management page.*
2. *The system displays capstone groups and related details.*

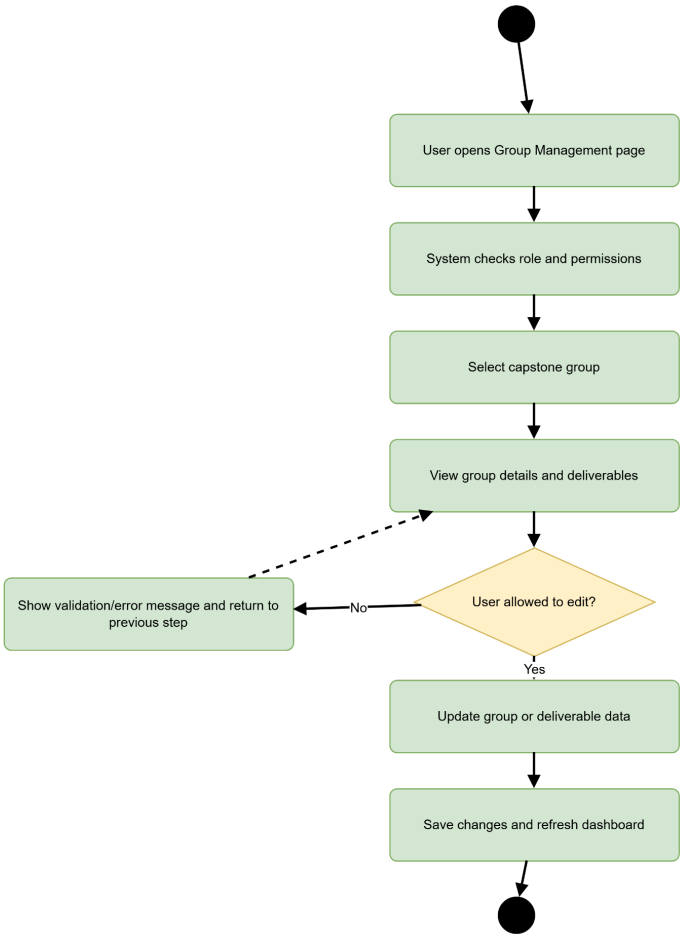
3. *The user selects a group.*
4. *The system displays group members, adviser, project title, software name, description, assigned deliverables, deadlines, remarks, and current submission statuses.*
5. *The Admin may edit group records and deliverable settings.*
6. *The Adviser may view assigned groups and monitor deliverables.*
7. *The system saves valid updates and refreshes the dashboard.*

Alternative Flow:
If a user tries to access a group outside their assigned permission, the system denies access and displays an authorization message.

Postconditions:
Capstone group and deliverable records are updated or displayed based on the user's role.

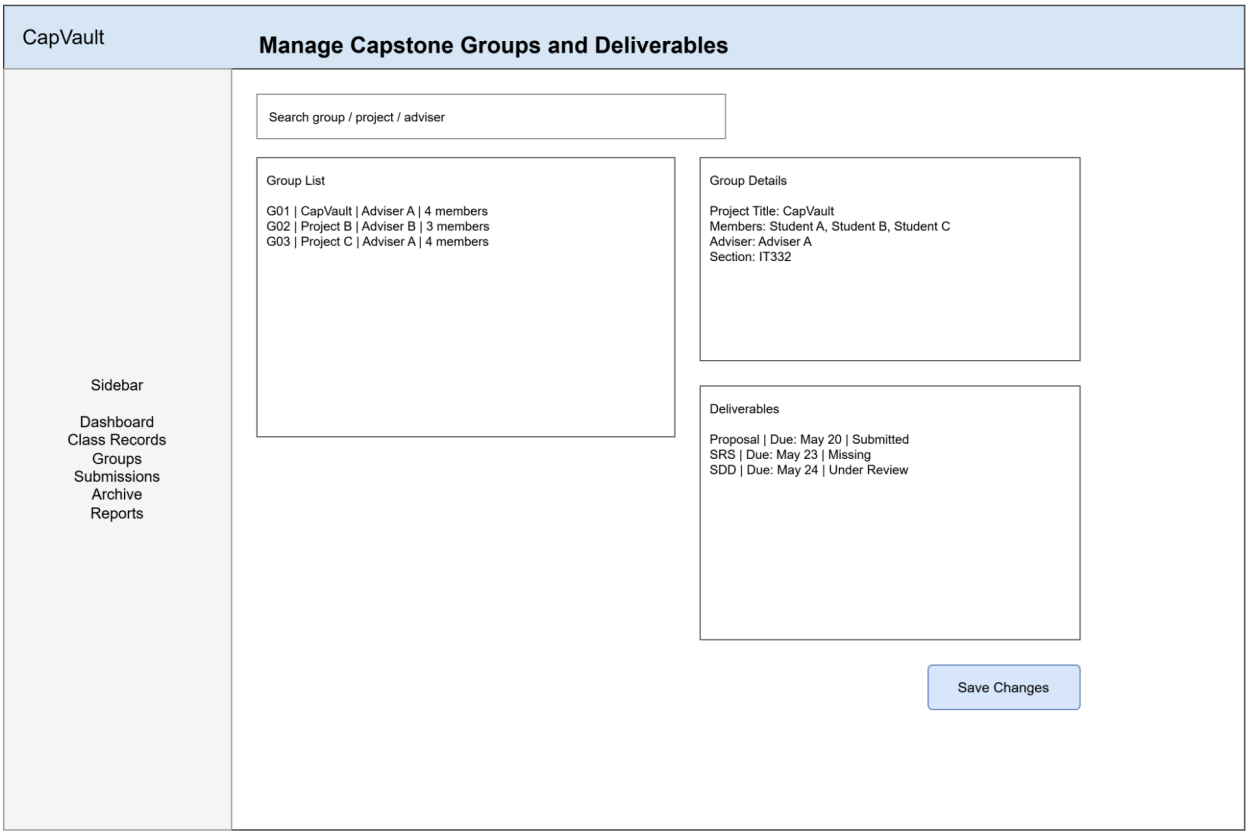
Activity Diagram:

Activity - 1.2 Manage Groups and Deliverables



Wireframe:

Wireframe - 1.2 Manage Groups and Deliverables

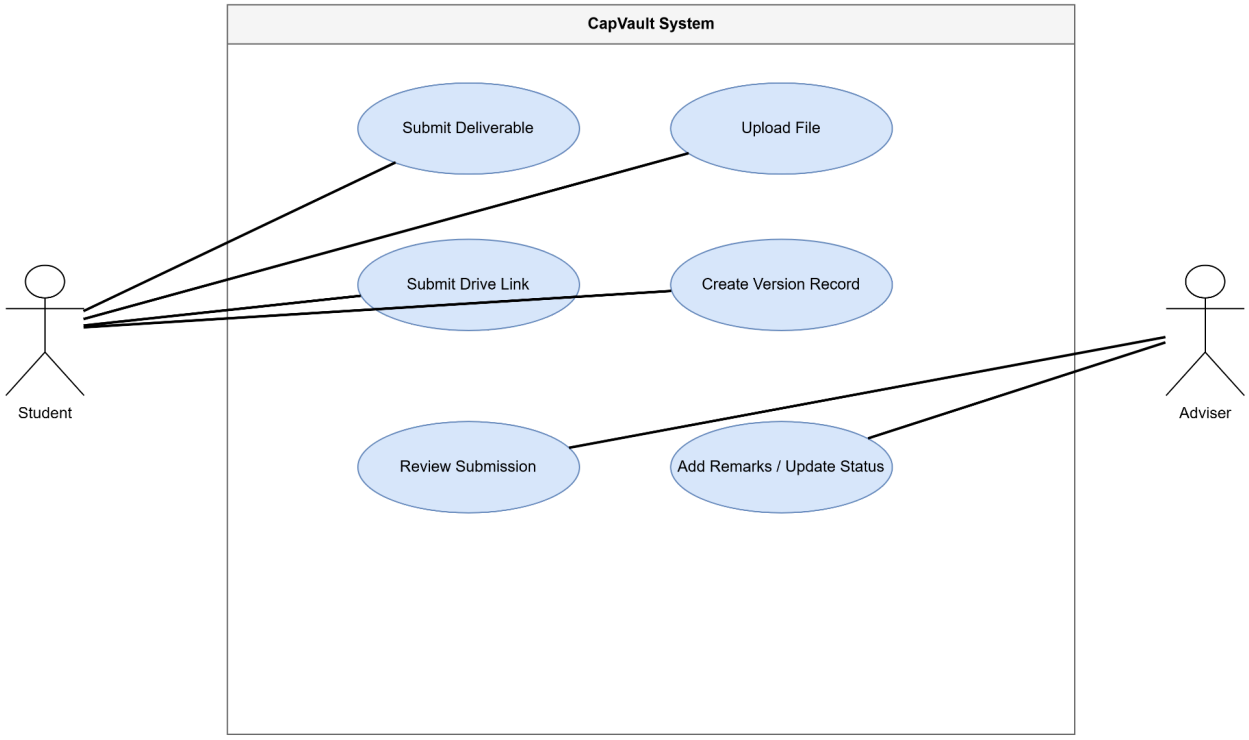


Module 2: Submission and Version Control

2.1 Submit Capstone Deliverable

Use Case Diagram:

UC - Module 2 Submission and Version Control



Use Case Description:

Use Case Name: *Submit Capstone Deliverable*
Primary Actor: *Student*
Supporting System: *Google Drive API*

Goal:
To allow students to submit required capstone deliverables through direct file upload or Google Drive link.

Preconditions:
The Student is logged in. The Student belongs to a registered capstone group. A deliverable is assigned to the group.

Main Flow:

1. *The Student opens the Submission page.*
2. *The system displays assigned deliverables and deadlines.*

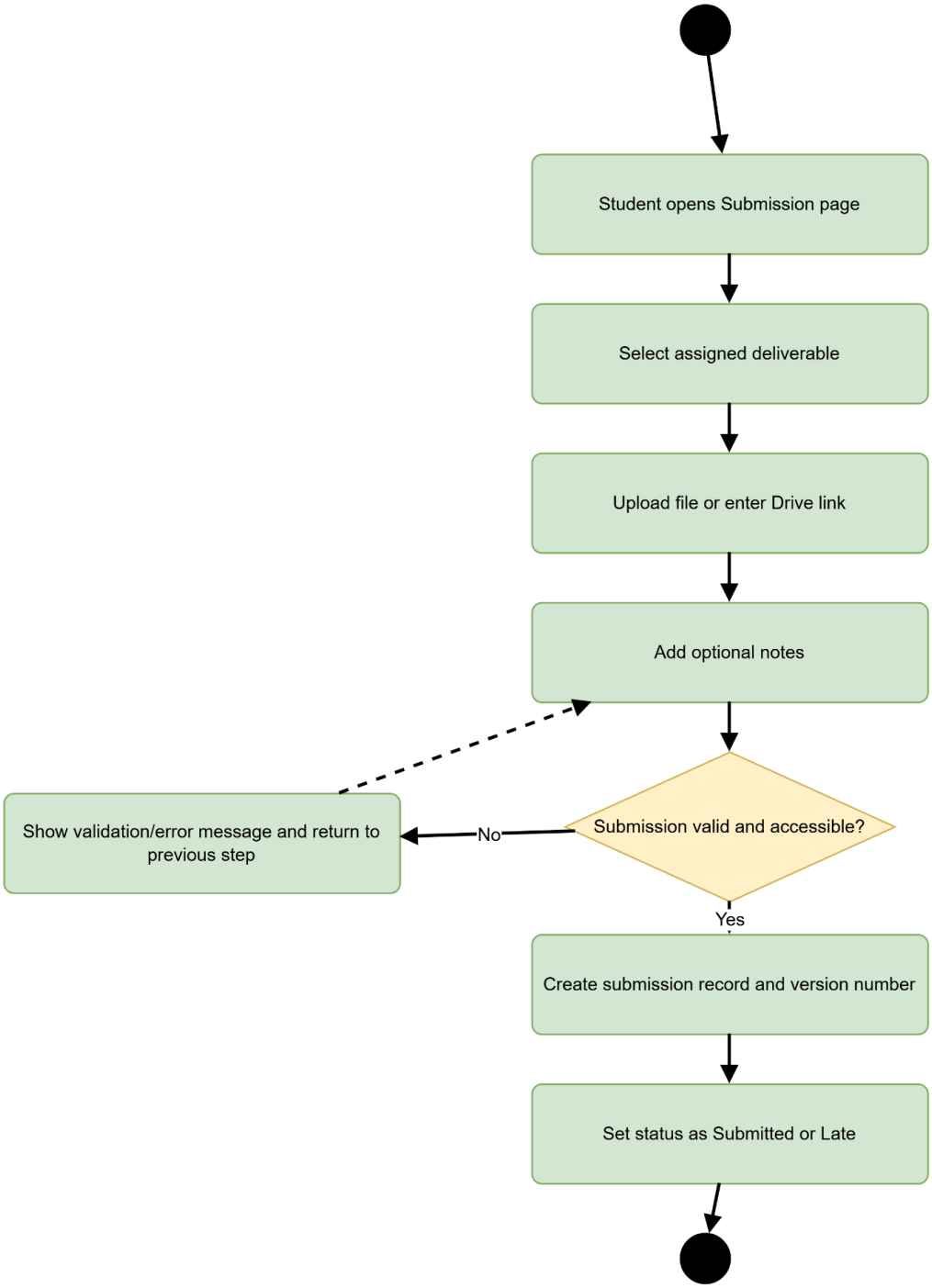
3. *The Student selects the deliverable type.*
4. *The Student uploads a file or enters a Google Drive link.*
5. *The Student adds optional notes.*
6. *The system validates the required fields and file or link accessibility.*
7. *The system records the submission details.*
8. *The system assigns a version number.*
9. *The system marks the submission as Submitted or Late depending on the deadline.*
10. *The system confirms successful submission.*

Alternative Flow:
If the file type is unsupported or the Google Drive link is inaccessible, the system displays an error and asks the Student to correct the submission.

Postconditions:
The submitted deliverable is recorded and prepared for review, versioning, and archival processing.

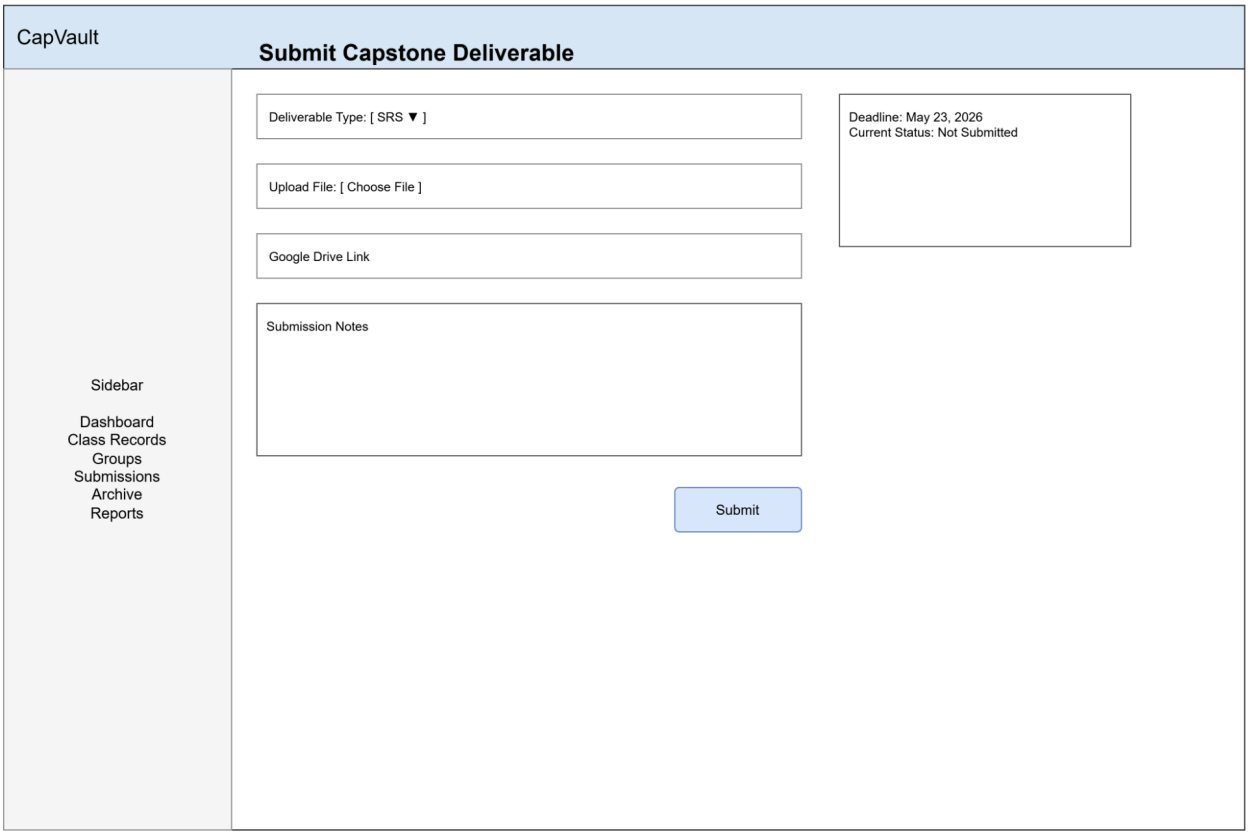
Activity Diagram:

Activity - 2.1 Submit Capstone Deliverable



Wireframe:

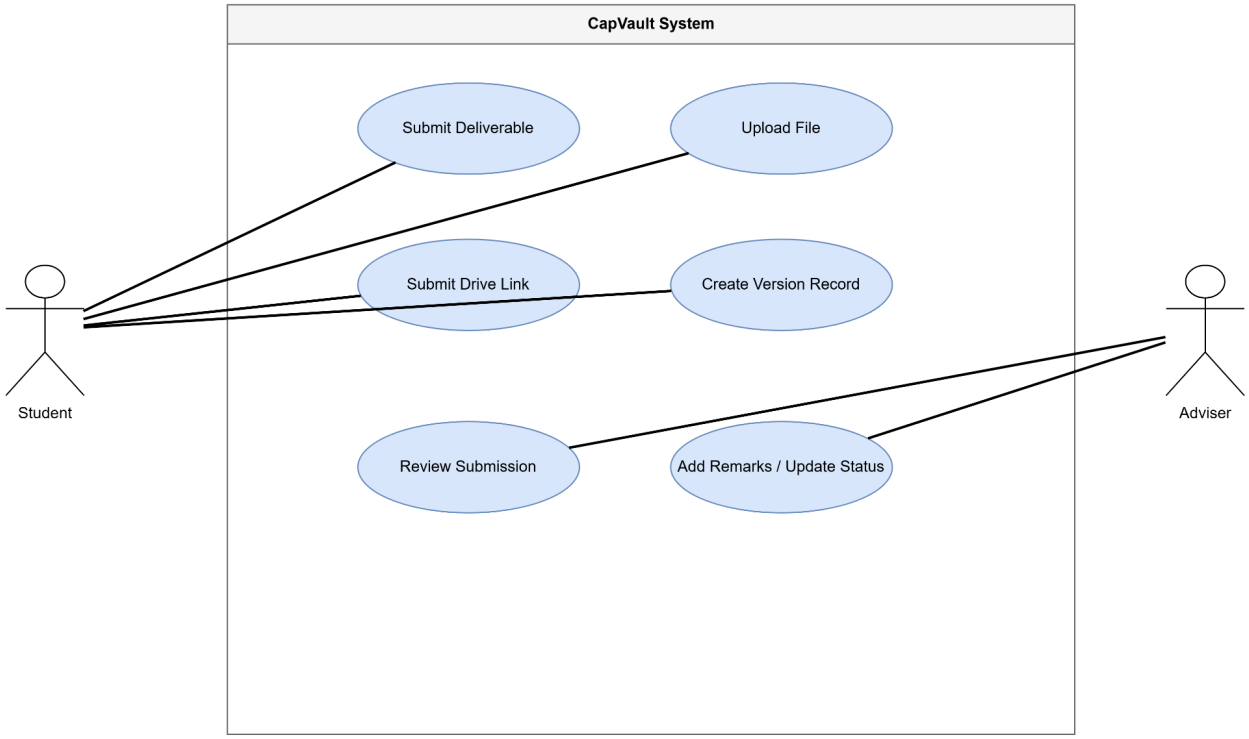
Wireframe - 2.1 Submit Deliverable



2.2 Review Submission and Add Remarks

Use Case Diagram:

UC - Module 2 Submission and Version Control



Use Case Description:

Use Case Name: *Review Submission and Add Remarks*
Primary Actor: *Adviser*
Secondary Actor: *Student*

Goal:
To allow advisers to review submitted capstone documents and provide remarks without overwriting previous versions.

Preconditions:
The Adviser is logged in. The submitted document belongs to a group assigned to the Adviser.

Main Flow:

1. *The Adviser opens the Submission Review page.*
2. *The system displays assigned groups and submitted deliverables.*
3. *The Adviser selects a submission.*
4. *The system displays document details, version history, metadata, and current status.*

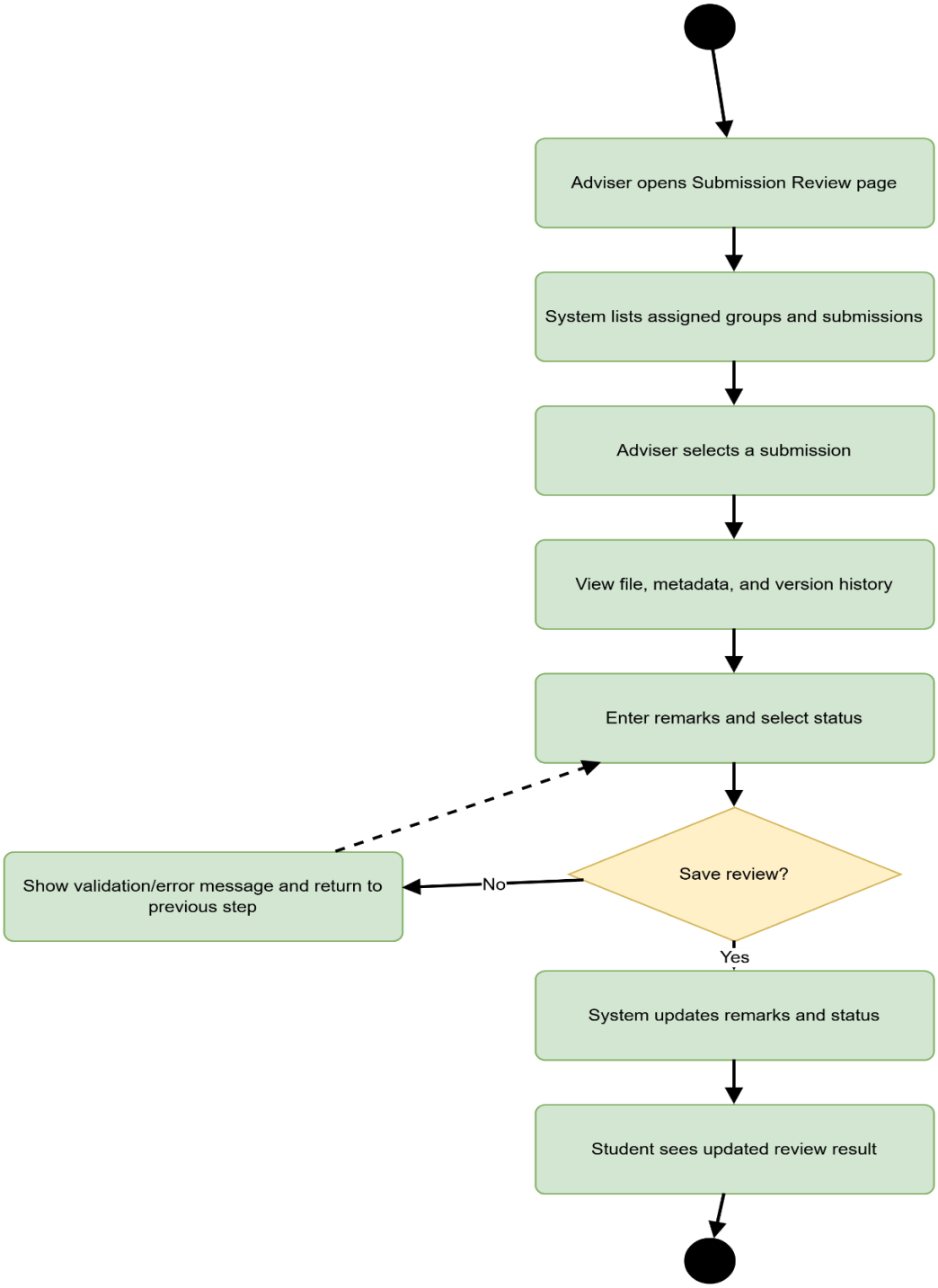
5. *The Adviser views or downloads the submitted file.*
6. *The Adviser enters remarks.*
7. *The Adviser updates the submission status to Under Review, Needs Revision, Approved, Rejected, or Final.*
8. *The system saves the remarks and status update.*
9. *The Student can view the updated remarks and status.*

Alternative Flow:
If the Adviser attempts to access an unassigned group, the system denies access.

Postconditions:
Submission remarks and status are updated while previous file versions remain preserved.

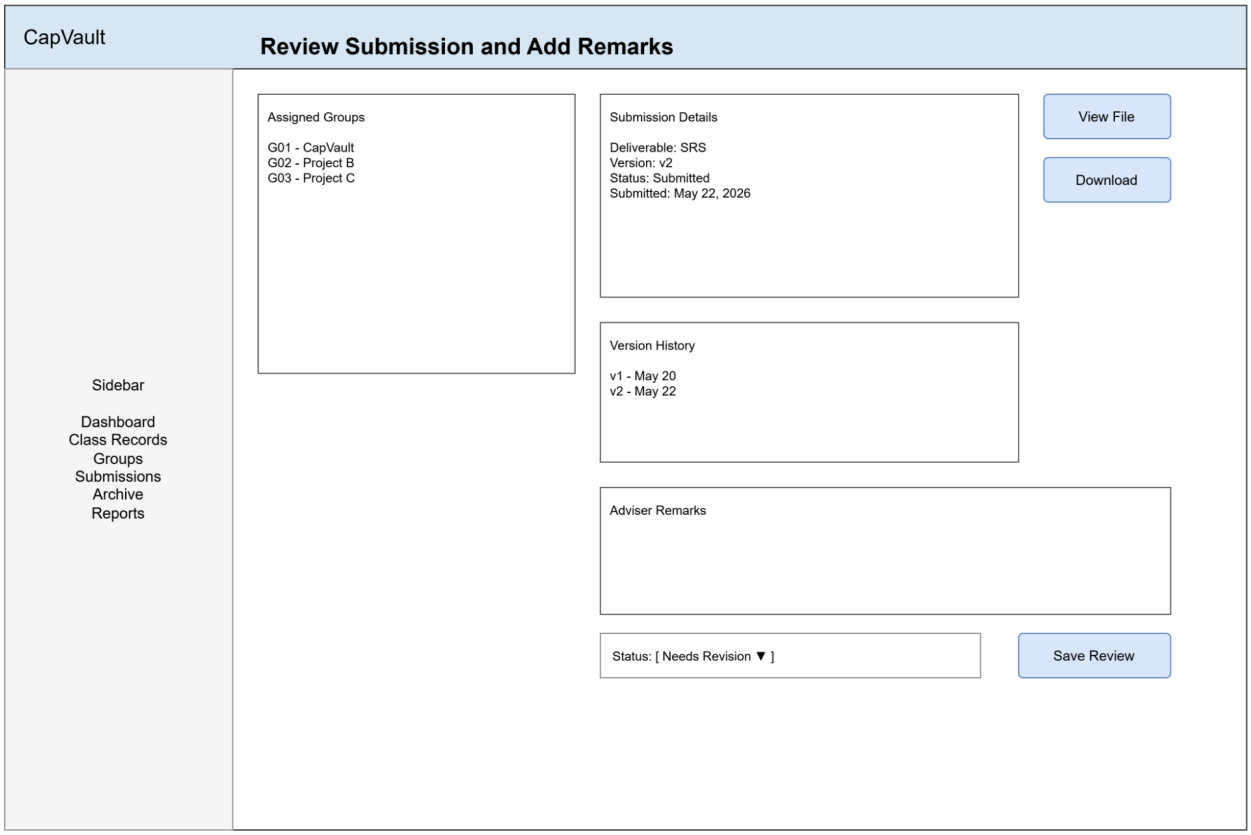
Activity Diagram:

Activity - 2.2 Review Submission and Add Remarks



Wireframe:

Wireframe - 2.2 Review Submission

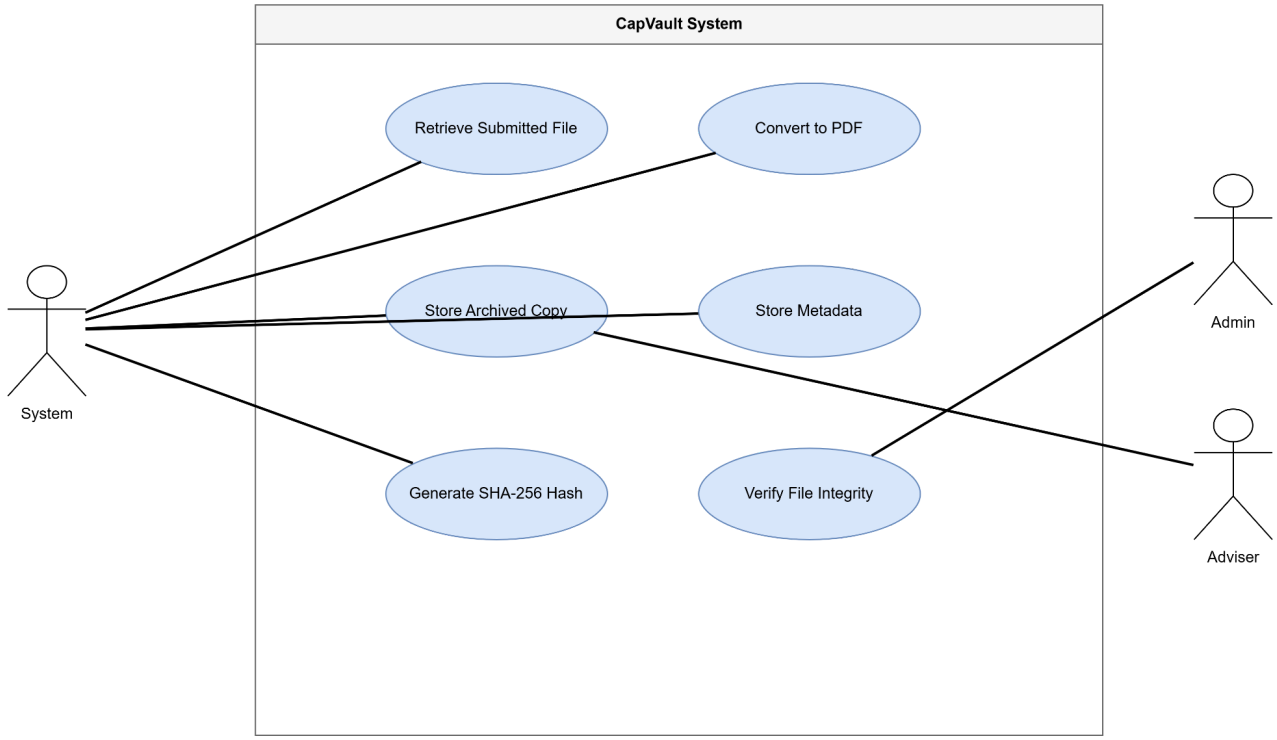


Module 3: Archival Vault and File Integrity Management

3.1 Archive Submitted Document

Use Case Diagram:

UC - Module 3 Archival Vault and Integrity



Use Case Description:

Use Case Name: *Archive Submitted Document*
Primary Actor: *System*
Secondary Actors: *Admin, Adviser*
Supporting Systems: *Google Drive API, PDF Conversion Service*

Goal:
To store an institutional archived copy of a submitted capstone document with metadata.

Preconditions:
A valid submission exists. The uploaded file or Google Drive link is accessible.

Main Flow:

1. *The system receives a valid submission.*
2. *The system retrieves the uploaded file or Drive-linked document.*

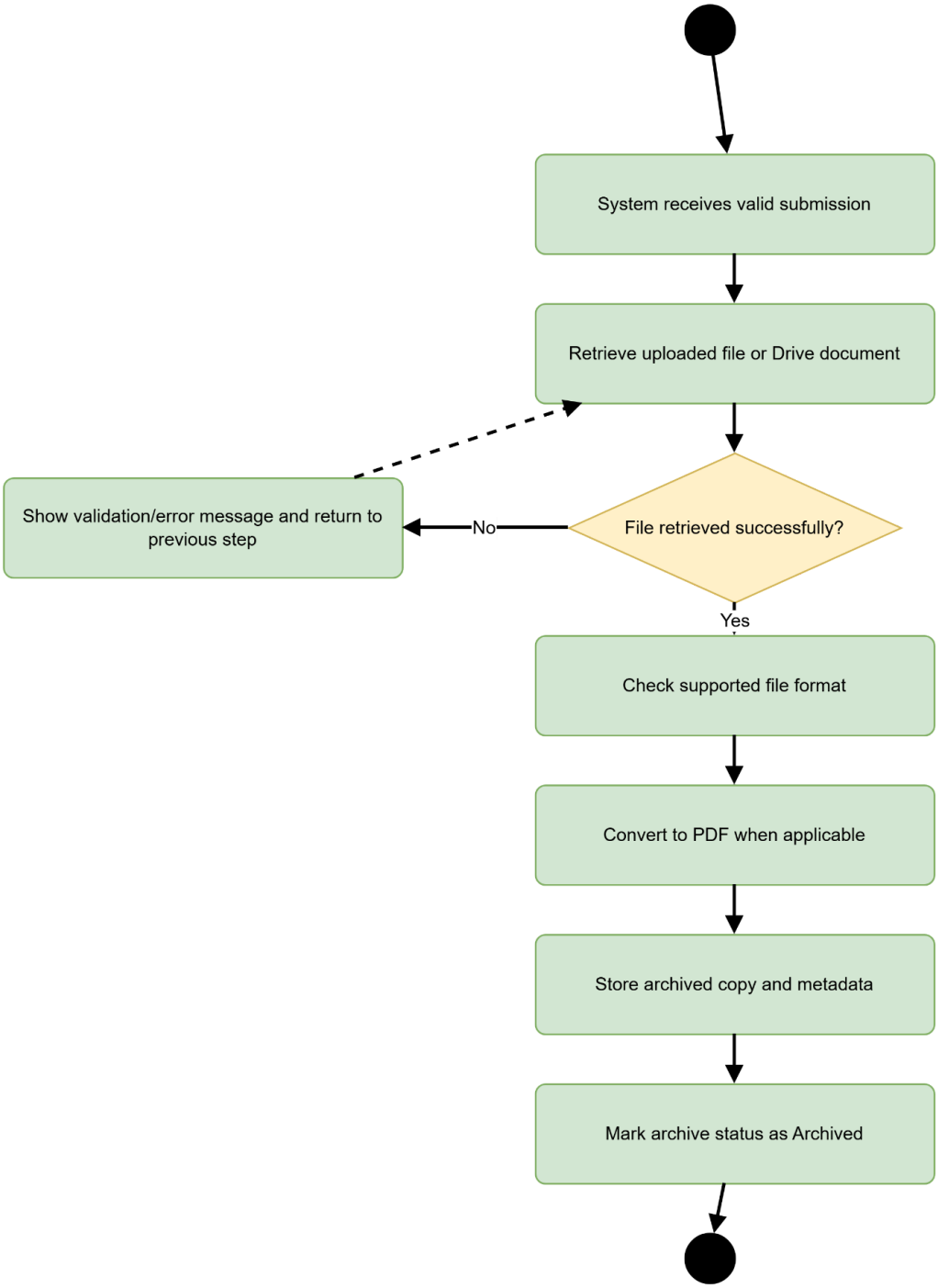
3. *The system checks whether the file format is supported for archival processing.*
4. *The system converts supported DOCX or Google Docs files into PDF when applicable.*
5. *The system stores the archived copy in file storage.*
6. *The system records archive metadata such as project title, group code, adviser, deliverable type, version number, submission date, archive date, and status.*
7. *The system confirms that the document has been archived.*

Alternative Flow:
If the file cannot be retrieved or converted, the system records the error and marks the archive attempt as failed or pending.

Postconditions:
An archived document copy and its metadata are stored in the system.

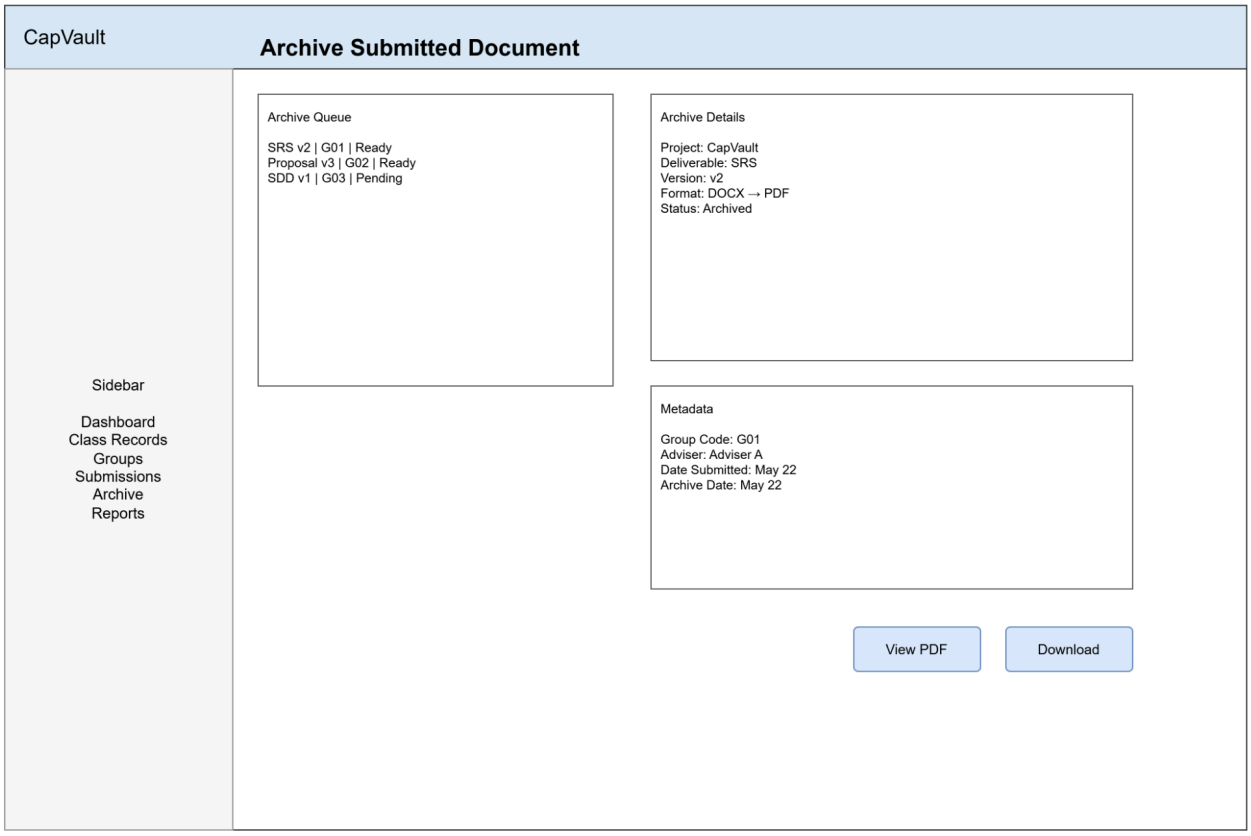
Activity Diagram:

Activity - 3.1 Archive Submitted Document



Wireframe:

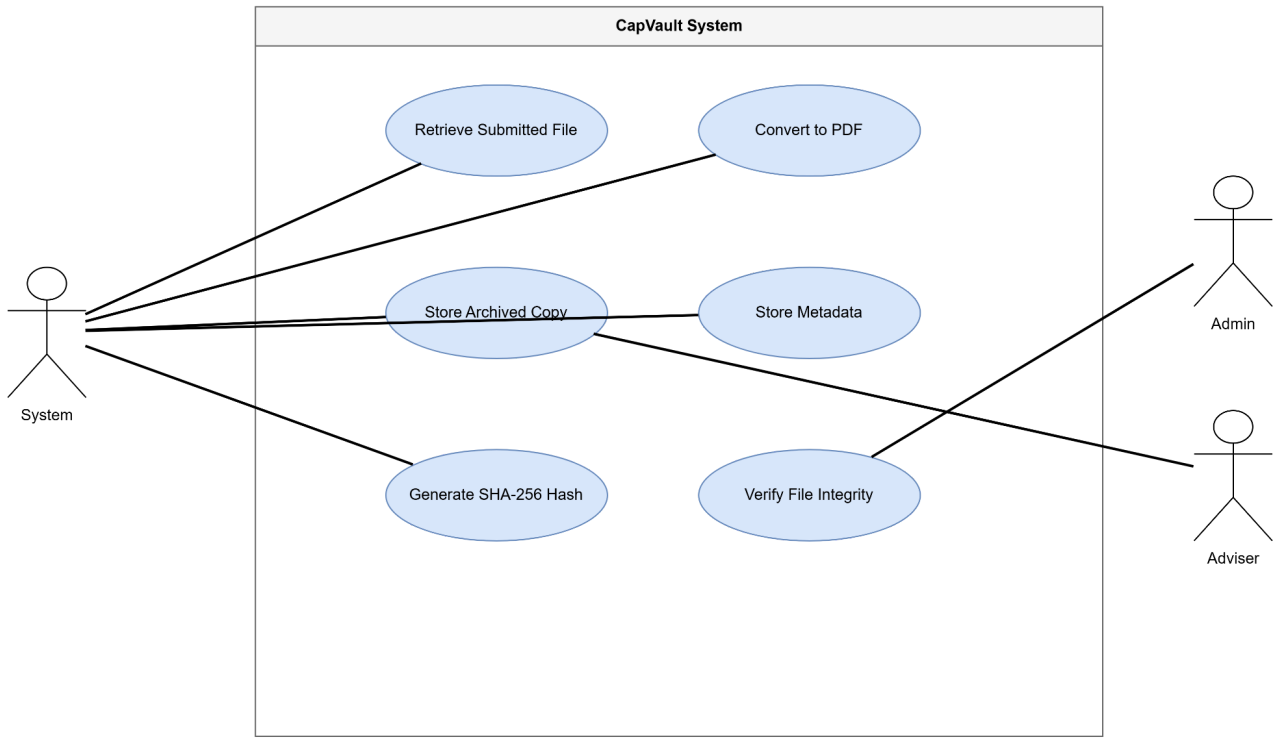
Wireframe - 3.1 Archive Document



3.2 Generate and Verify SHA-256 Hash

Use Case Diagram:

UC - Module 3 Archival Vault and Integrity



Use Case Description:

Use Case Name: *Generate and Verify SHA-256 Hash*
Primary Actor: *System*
Secondary Actor: *Admin*

Goal:
To generate and compare SHA-256 hashes for archived documents to support file integrity verification.

Preconditions:
A document has been submitted or archived. The file is accessible to the system.

Main Flow:

1. *The system reads the archived file.*
2. *The system generates a SHA-256 hash value.*
3. *The system stores the hash value with the document metadata.*
4. *When verification is needed, the system regenerates the hash from the stored file.*
5. *The system compares the new hash with the stored hash.*

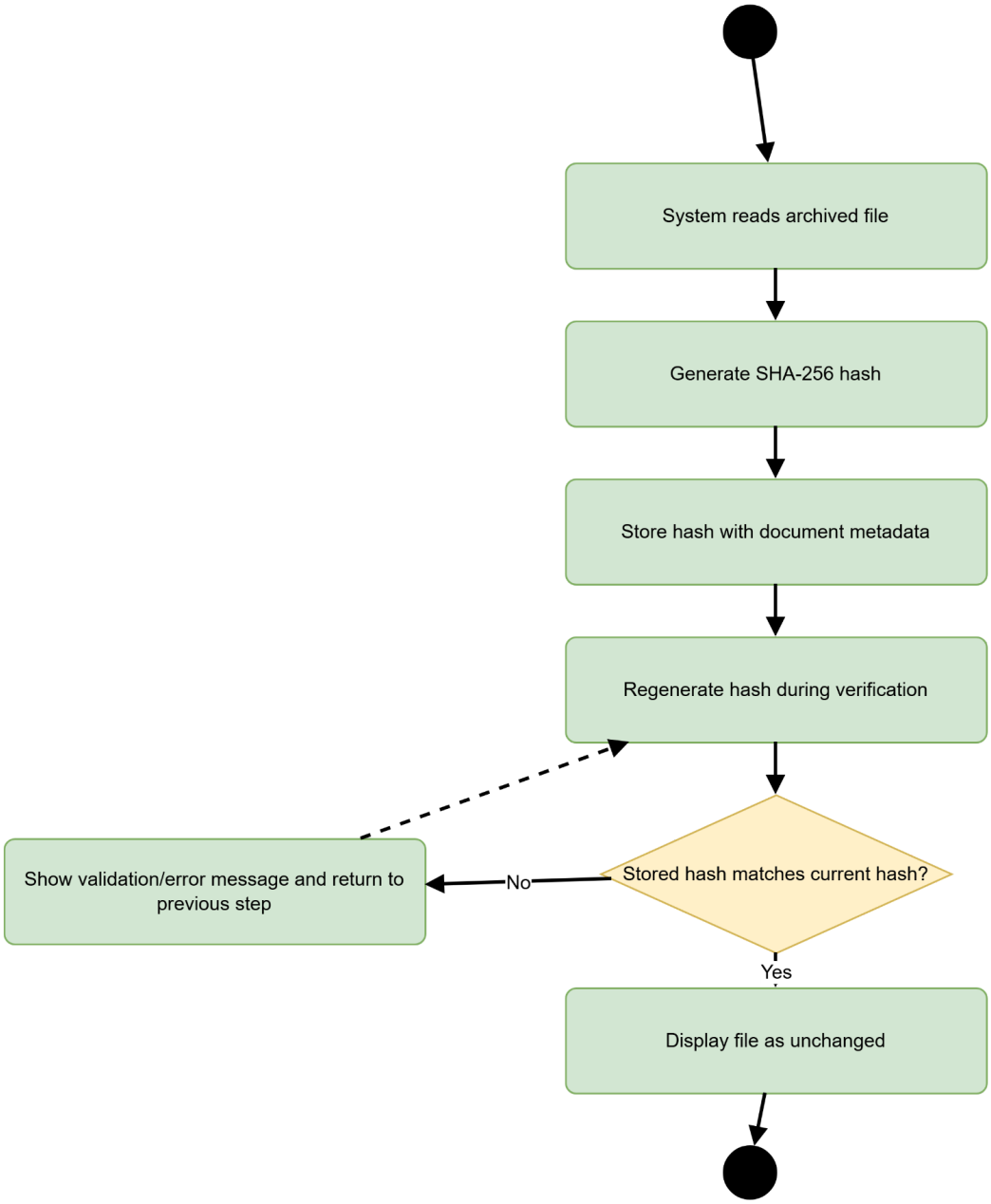
6. *The system displays whether the file is unchanged or modified.*

Alternative Flow:
If the file is missing or unreadable, the system displays an integrity verification error.

Postconditions:
The archived file has a stored hash and can be verified for changes.

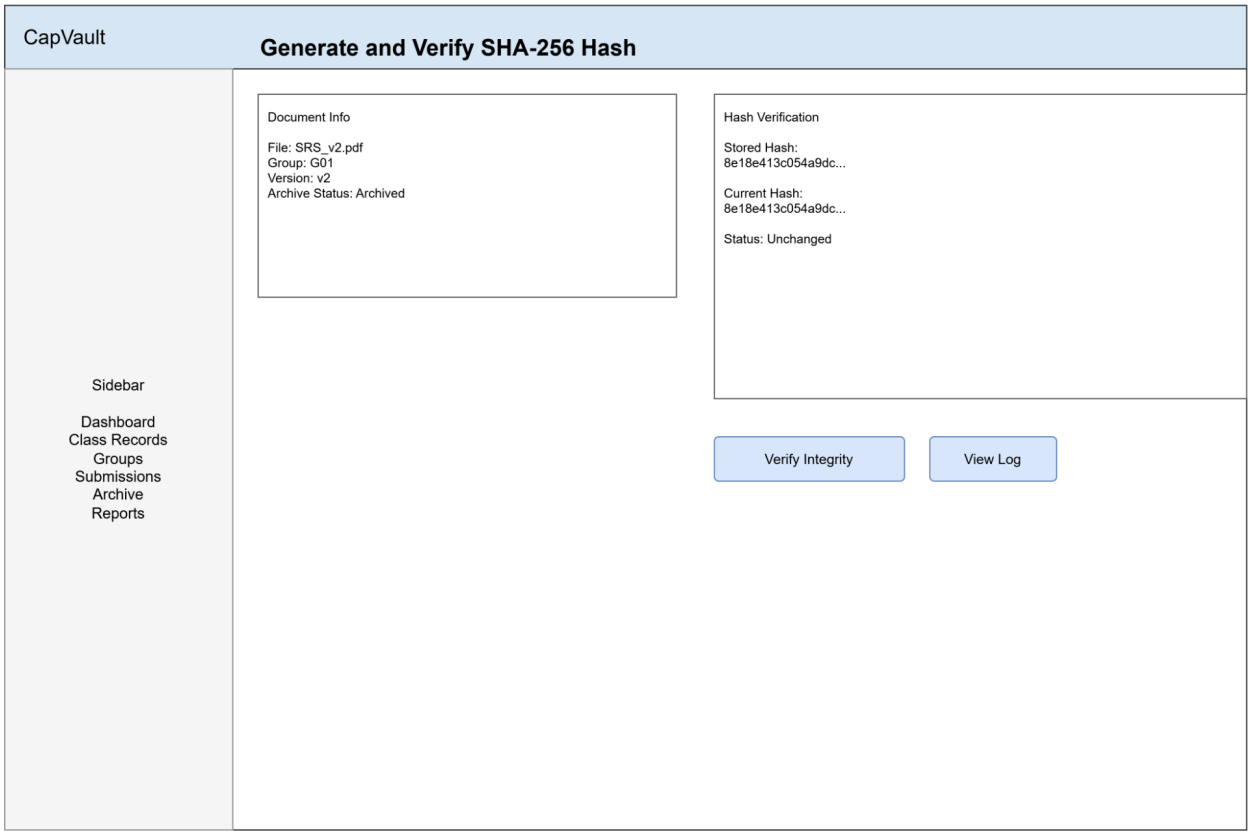
Activity Diagram:

Activity - 3.2 Generate and Verify SHA-256 Hash



Wireframe:

Wireframe - 3.2 Verify SHA-256 Hash

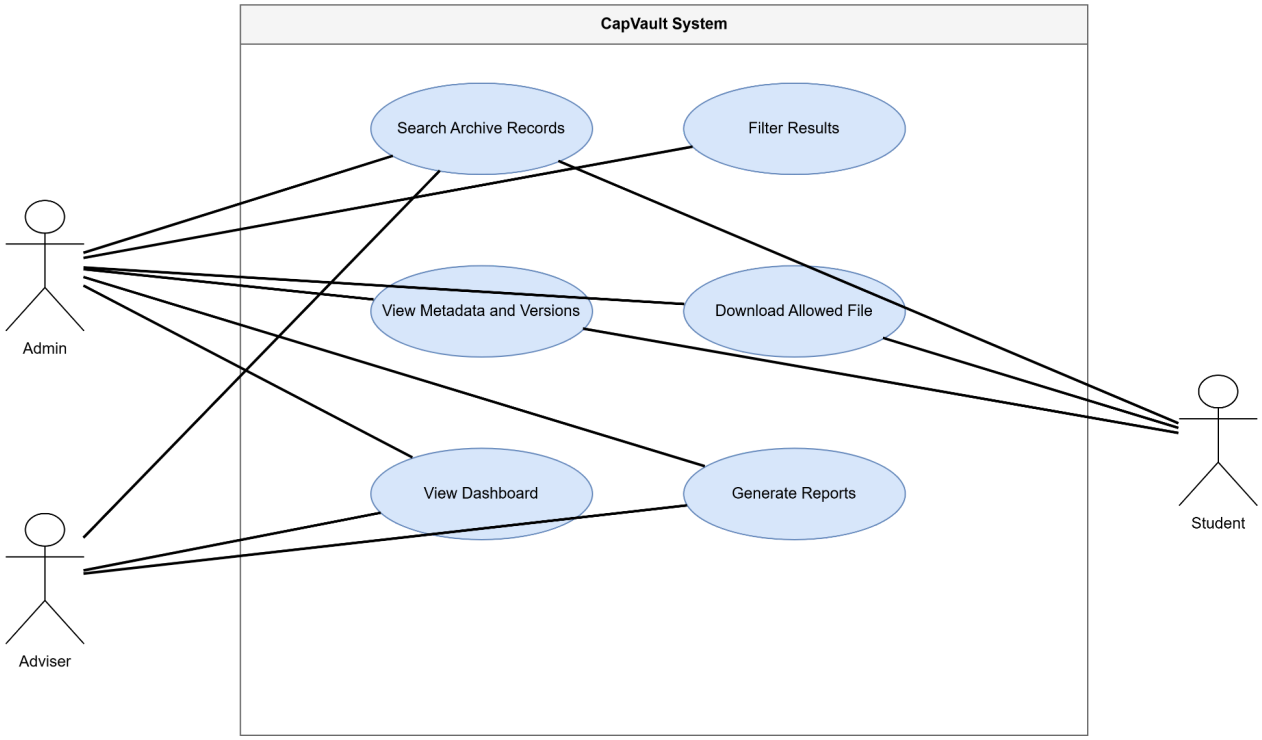


Module 4: Search, Retrieval, and Reporting

4.1 Search and Retrieve Archived Records

Use Case Diagram:

UC - Module 4 Search Retrieval Reporting



Use Case Description:

Use Case Name: *Search and Retrieve Archived Records*
Primary Actors: *Admin, Adviser, Student*

Goal:
To allow authorized users to search, filter, view, and download archived capstone documents.

Preconditions:
The user is logged in. The user has permission to access the requested records. Archived records exist in the system.

Main Flow:

1. *The user opens the Archive Search page.*
2. *The user enters search keywords or selects filters.*
3. *The system checks the user's role and access permissions.*

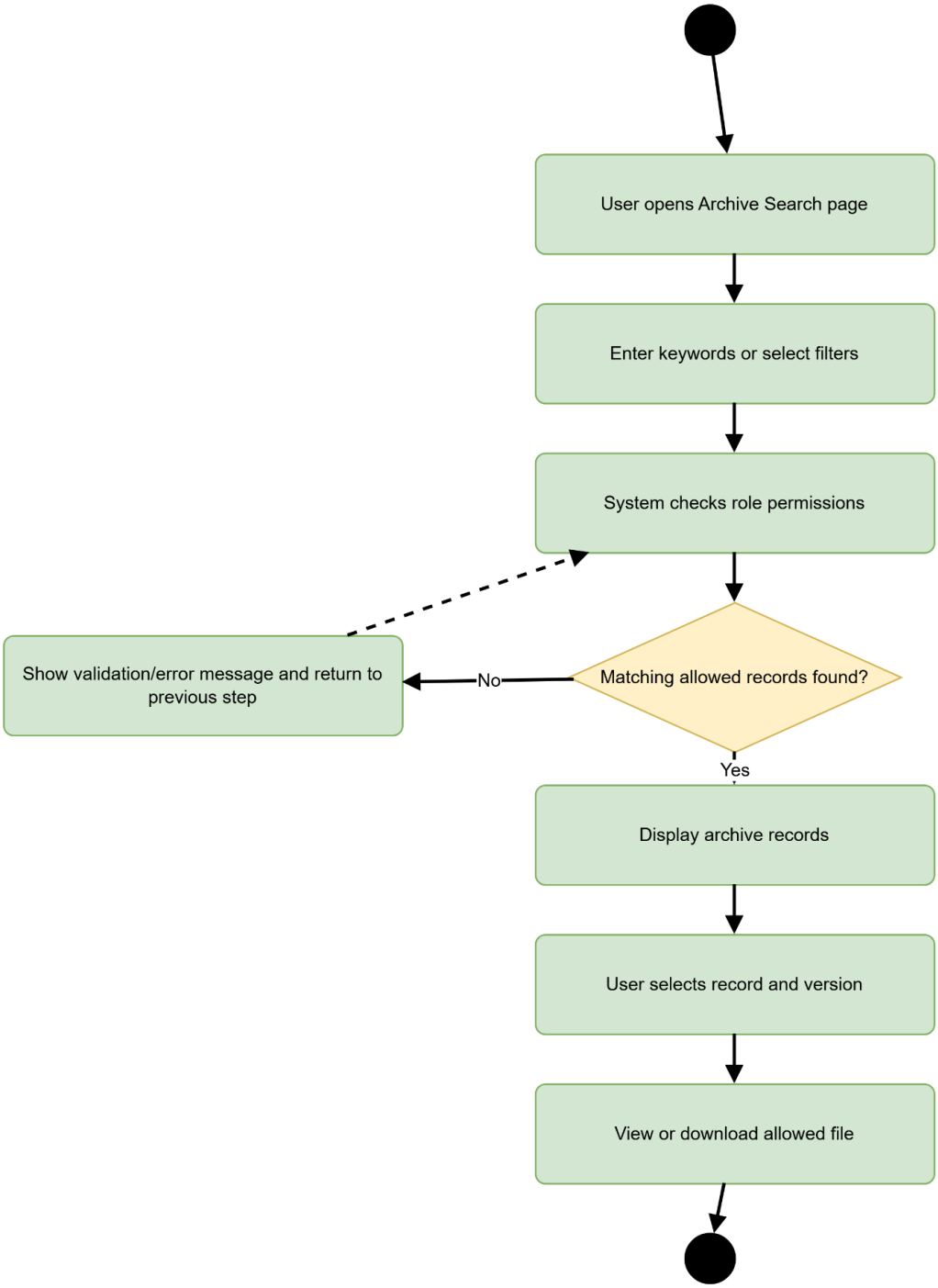
4. *The system displays matching archive records.*
5. *The user selects a record.*
6. *The system displays document metadata and available versions.*
7. *The user views or downloads the allowed file.*

Alternative Flow:
If no matching record is found, the system displays a no-results message. If the user lacks permission, the system hides or blocks restricted records.

Postconditions:
The user retrieves or views the selected archive record based on role permissions.

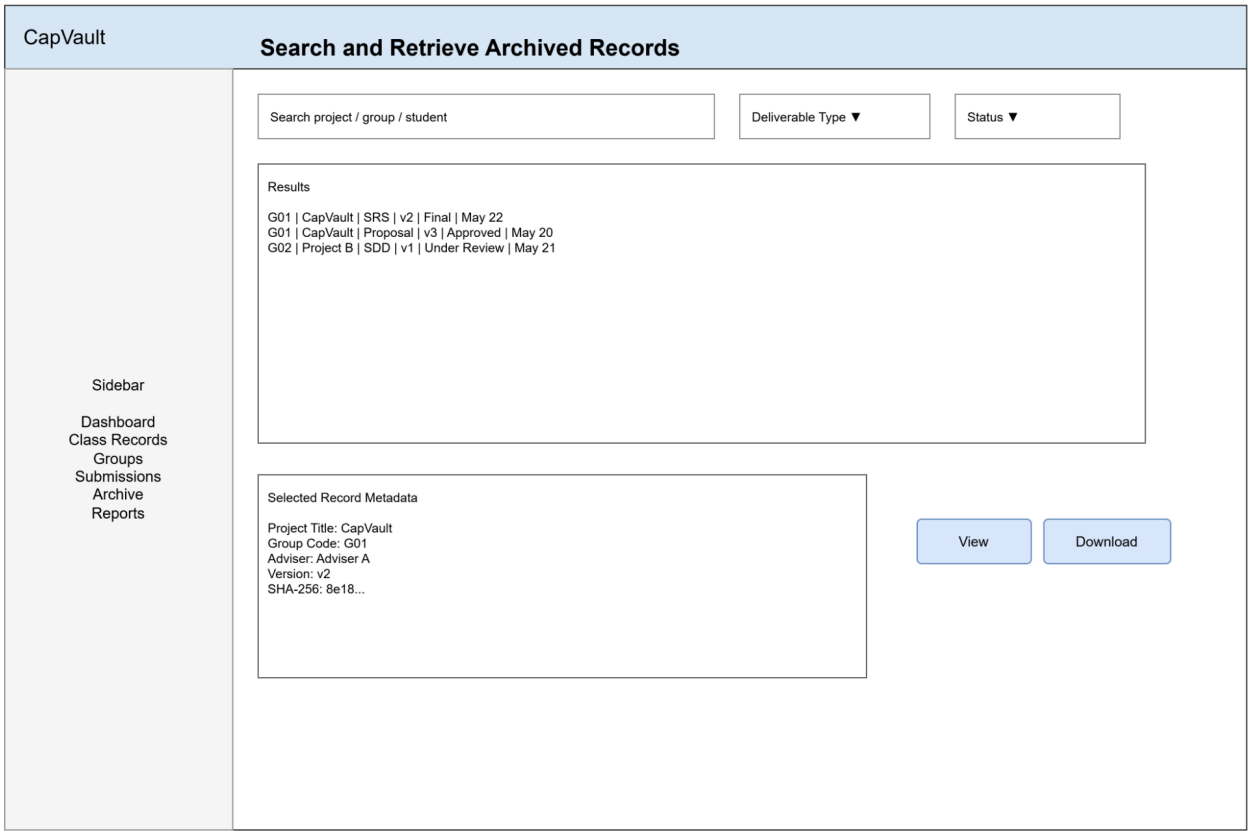
Activity Diagram:

Activity - 4.1 Search and Retrieve Archived Records



Wireframe:

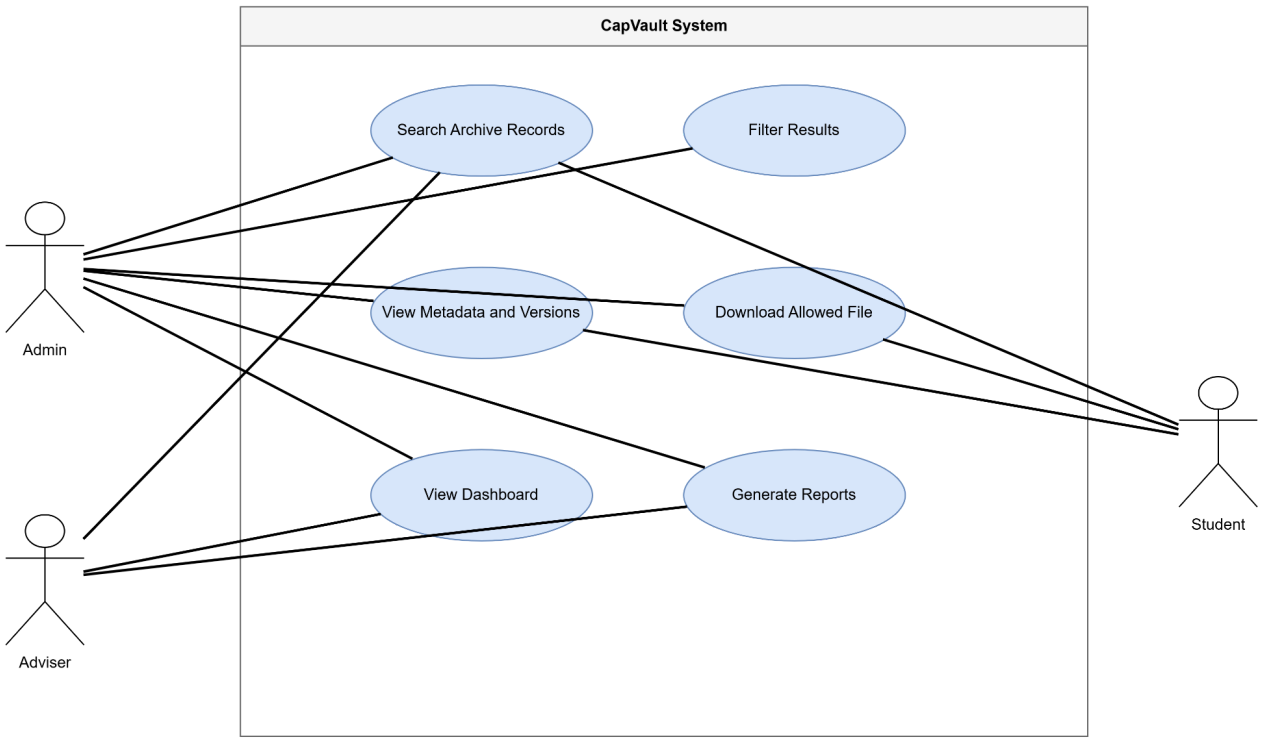
Wireframe - 4.1 Search Records



4.2 Generate Dashboard and Reports

Use Case Diagram:

UC - Module 4 Search Retrieval Reporting



Use Case Description:

Use Case Name: *Generate Dashboard and Reports*
Primary Actors: *Admin, Adviser*

Goal:
To display summary counts and reports for submitted, missing, late, needs revision, approved, final, and archived deliverables.

Preconditions:
The user is logged in as Admin or Adviser. Capstone group and submission records exist.

Main Flow:

1. *The user opens the Dashboard or Reports page.*
2. *The system retrieves group, deliverable, submission, and archive records based on the user role.*
3. *The system calculates counts and summaries for submission and archive status.*
4. *The system displays dashboard cards, tables, and report filters.*
5. *The user selects report filters such as section, adviser, group, deliverable type, or date.*

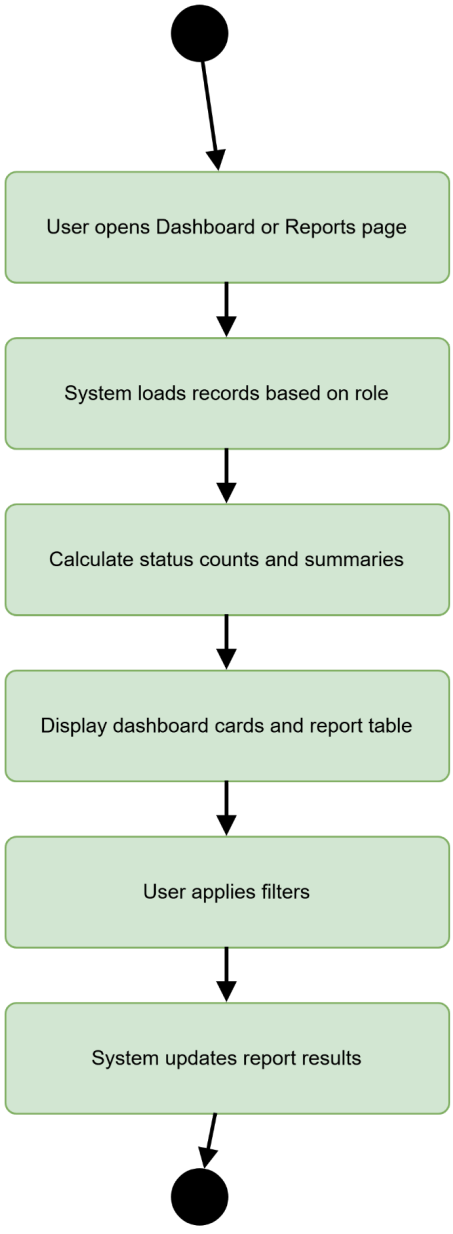
6. *The system updates the report results.*

Alternative Flow:
If there are no records, the system displays empty dashboard values and a no-records message.

Postconditions:
The user views updated progress and archive reports.

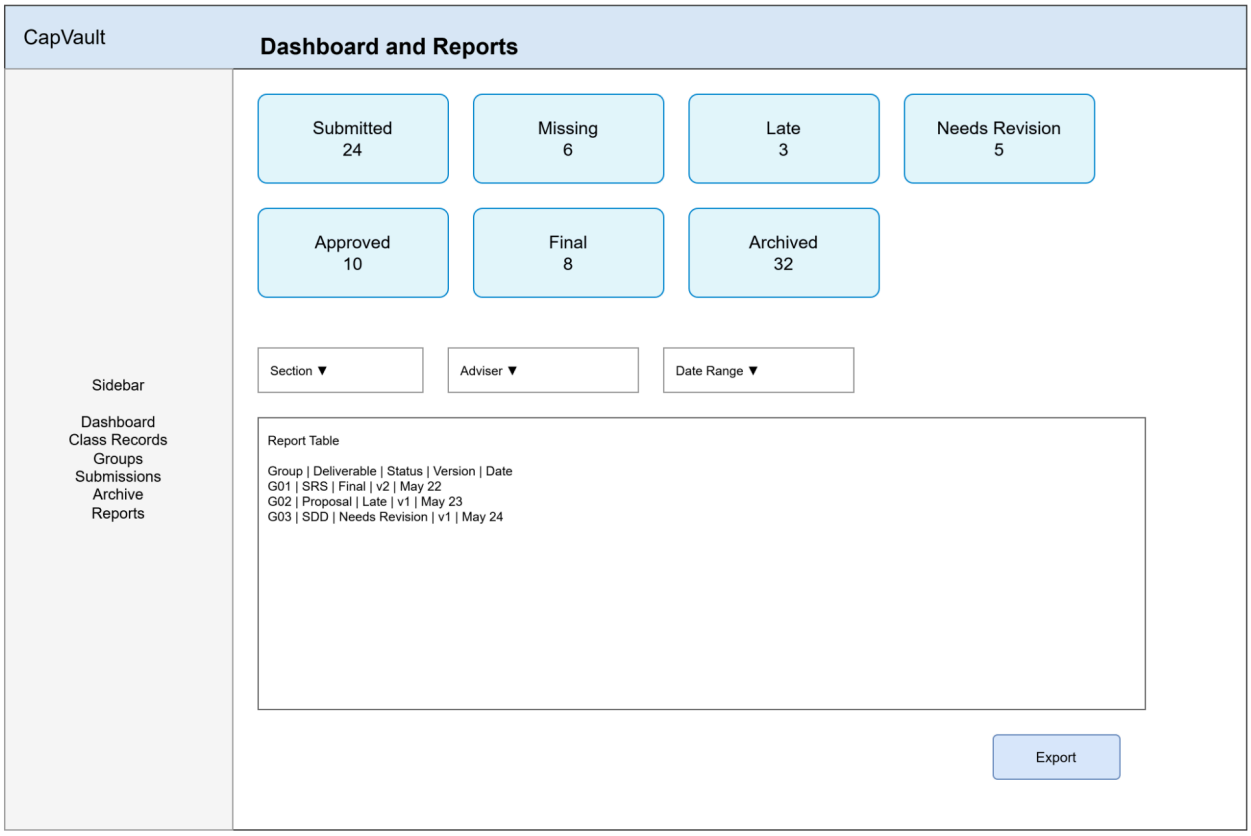
Activity Diagram:

Activity - 4.2 Generate Dashboard and Reports



Wireframe:

Wireframe - 4.2 Dashboard and Reports



3.4 Non-functional requirements

Performance

- The system shall load the main dashboard within **5 seconds** under normal test conditions using the selected capstone class or sample dataset.
- The system shall retrieve search results within **5 seconds** when filtering archived records by project title, group code, student name, adviser, section, deliverable type, status, version, or submission date.
- The system shall reduce adviser submission status checking time by at least **30%** compared to manual checking through separate Google Sheets, Drive links, and message records.
- The system shall reduce archived document retrieval time by at least **40%** compared to manual searching through Google Drive folders, spreadsheet links, or separate document records.
- The system shall successfully archive and retrieve at least **90%** of supported test documents during controlled test cases.
- The system shall process standard file uploads and Drive-linked documents within a reasonable time depending on file size, internet connection, server performance, and Google API response time.
- The system shall support simultaneous access by Admin, Adviser, and Student users during normal class-level usage without major delays in dashboard loading, submission viewing, or archive searching.

Security

- The system shall require users to log in through **Google account validation** before accessing any system function.
- The system shall enforce **role-based access control** for Admin, Adviser, and Student users.
- The system shall restrict Students to their own group records, submission history, adviser remarks, and allowed archive records.
- The system shall restrict Advisers to assigned capstone groups, submitted deliverables, review pages, remarks, and related archive records.
- The system shall allow Admin users to manage class records, user roles, system settings, archive records, and overall reports.
- The system shall prevent unauthorized users from viewing, downloading, editing, or deleting restricted capstone documents and archive records.
- The system shall record important actions in activity logs, including submissions, updates, adviser remarks, status changes, archive actions, file views, and downloads.
- The system shall use secure communication during deployment, preferably **HTTPS**, to protect login sessions, document transactions, and API communication.
- The system shall store SHA-256 hash values for archived documents to support file integrity verification and detect changed files.
- The system shall not expose Google API credentials, database credentials, authentication tokens, or server configuration details to unauthorized users.

Reliability

- The system shall preserve **100%** of submitted document versions during controlled test cases by saving updated submissions as new versions instead of overwriting previous files.
- The system shall maintain submission records, version records, archive metadata, and activity logs even if a later Google Drive link becomes inaccessible, as long as an institutional archived copy was successfully stored.
- The system shall display clear error messages when file upload, Google Drive retrieval, PDF conversion, archive storage, or hash verification fails.
- The system shall mark failed archive attempts as **Failed** or **Pending** instead of silently losing the submitted record.
- The system shall allow users to retry failed document retrieval or archival processing when the issue is caused by temporary connection, permission, or API problems.
- The system shall verify archived files by regenerating and comparing SHA-256 hash values during integrity checks.
- The system shall keep archived document metadata connected to the correct project, group, deliverable, version, adviser, and submission status.
- The system shall continue to provide access to stored class records, submission records, and archived metadata even if Google Sheets or Google Drive access is temporarily unavailable.
- The system shall protect archived records from accidental replacement by requiring updated submissions to be stored as separate document versions.